

Open-Weight, Prompt-Driven Melody-to-Lyric Models: Landscape & Recommendations

TL;DR

- No open-weight model exists today that natively accepts BOTH a symbolic note sequence AND a controllable theme/style prompt to produce melody-aligned lyrics in a single trained mode. The closest open-weight system, **SongComposer** (InternLM2-7B, Apache-2.0 code + freely usable weights), accepts symbolic note tuples (pitch, duration, rest) and produces aligned lyrics, but its melody-to-lyric (M2L) task uses only a fixed "harmonize with this melody" instruction with no trained theme/topic conditioning — though its single-string prompt API lets you experimentally concatenate a theme.
- The field splits cleanly into four buckets — (a) lyric→melody, (b) melody→lyric, (c) joint symbolic song models, (d) end-to-end audio song generators that take lyrics+style as input (the inverse of what you want). Your exact target (note sequence + prompt → lyrics) sits between (b) and (c) and is only partially served.
- The pragmatic path is **SongComposer for symbolic melody alignment** (self-hostable on a 3090/A6000, fine-tunable to add theme conditioning), or a **custom constrained-decoding pipeline** (Lyra-style: an open LLM generates a content-planned draft, then a syllable/note-count constraint compiler aligns it to your note sequence) when you want full prompt controllability with permissive licensing.

Key Findings

Exact-match status. Across the 2021–2026 literature, melody-to-lyric (M2L) generation is well-studied, and prompt-controllable lyric generation is well-studied, but the intersection — a downloadable model that ingests an annotated note sequence *and* a free-text theme/style prompt and emits melody-fitted lyrics in one supported inference mode — does not exist as a packaged open-weight artifact. The systems that come closest each miss one axis: either they take melody but not a controllable theme (SongComposer M2L, SongMASS), or they take a theme/content control but compile the melody as inference-time decoding constraints rather than as a model input (Lyra), or they are research code without released weights (REFFLY).

SongComposer is the single best open-weight starting point. It is an InternLM2-7B-based LLM ("We adopt InternLM2-7B as our base model and set the maximum token length as 5120") that represents melody as text tuples `(word, <pitch>, <duration>, <rest>)` (e.g., `(可, <D4>, <137>, <79>)`), where pitch is scientific notation (C3-B5 vocal range), and duration/rest are 512 log-spaced discretized bins ($512 \approx 6$ s, $79 \approx$ near-zero), NOT milliseconds. It supports melody-to-lyric, lyric-to-melody, song continuation, and text-to-song in one model. Weights are on HuggingFace (`Mar2Ding/songcomposer_sft` and `Mar2Ding/songcomposer_pretrain`); code is Apache-2.0 and weights are stated to allow free commercial usage. However, per close inspection of the paper appendices and inference code, the M2L task was trained only with fixed harmonize-style instructions; theme/style/pace control lives in the separate text-to-song task (which generates melody from scratch rather than accepting one). The inference API is a single free-form string, so melody + theme can be concatenated experimentally, but that combined mode is untrained and unevaluated. [Hugging Face](#) [Hugging Face](#)

The four-bucket landscape.

- **(a) Lyric → Melody (the inverse direction on the symbolic side):** SongMASS, TeleMelody, ReLyMe, SongGLM, CSL-L2M. These take lyrics and emit notes. Not your target.
- **(b) Melody → Lyric:** SongMASS (bidirectional), Lyra (unsupervised, content-controllable), REFFLY (melody-constrained revision), SeqGAN-based melody-conditioned generators. This is the closest functional bucket.
- **(c) Joint symbolic song models:** SongComposer (lyrics+melody together, symbolic). Best fit for "symbolic melody in, lyrics out."
- **(d) End-to-end audio song generation (the popular inverse):** YuE, ACE-Step, DiffRhythm, SongGen. These take lyrics + a style/genre text prompt and emit *audio*. They are the opposite of what you want — lyrics are an input, not an output, and melody is not a symbolic input.

Details

Bucket (c): Joint symbolic — closest to exact match

SongComposer (arXiv 2402.17645, ACL 2025 Main; PJLab/CUHK/Shanghai AI Lab).

- *Inputs it actually takes*: a single free-form instruction string. For M2L, that string contains a fixed instruction ("Write lyrics that harmonize with this melody" and four equivalents) followed by a pure-melody representation: line-structured note tuples of (pitch, duration-bin, rest-bin). For text-to-song, it takes a theme/style/pace description like "Create a song on brave and sacrificing with a rapid pace." [Hugging Face](#)
- *Outputs*: symbolic lyrics aligned word-by-word to notes (M2L), or full melody+lyric sheets (other tasks).
- *Open weights / license*: Yes — [Mar2Ding/songcomposer_sft](#), [Mar2Ding/songcomposer_pretrain](#) on HuggingFace. Code Apache-2.0; weights "fully open for academic research and also allow free commercial usage." [Hugging Face](#) [Hugging Face](#)
- *Model size*: 7B (InternLM2-7B base; vocabulary extended with 120 pitch tokens + 512 duration-bin tokens), max token length 5120. Pure-lyric pretraining used 283K song lyrics (150K English + 133K Chinese) plus ~20K pure melodies and ~15K paired lyric-melodies. [arxiv](#)
- *Self-hosting*: fits comfortably on a single RTX 3090 (24 GB) at fp16/half precision; trivially on an A6000 (48 GB). Standard HF [transformers](#) [AutoModel](#) with [trust_remote_code=True](#).
- *Fit to requirement*: ~70%. It natively does symbolic-melody → aligned-lyrics. The missing 30% is trained theme/topic conditioning *while* a melody is supplied. The clean fix is QLoRA/LoRA fine-tuning on (melody + theme → lyrics) triples, which the architecture and single-string API directly support. This is the most promising adaptation target for your homelab.

Bucket (b): Melody → Lyric

SongMASS (AAAI 2021, arXiv 2012.05168, Microsoft Muzic). Masked seq2seq pretraining + attention-based alignment; supports both L2M and M2L. Melody is symbolic (pitch/duration tokens). It was trained on the Lakh MIDI Dataset (176,581 MIDI files; 65,954 melody tracks extracted via Midi-miner, pitches octave-shifted to MIDI 60–71). Code is open (github.com/microsoft/muzic/tree/main/songmass). It is a from-scratch fairseq model, not a prompt-controllable LLM — no theme/topic input; alignment is done via dynamic programming at inference. Fit: ~45% (melody in, lyrics out, but no text prompt controllability and dated text quality). [arXiv + 2](#)

Lyra ("Unsupervised Melody-to-Lyric Generation," ACL 2023, Amazon Science). The most relevant *design* for your needs: a hierarchical plan-and-write pipeline built on pretrained LMs that disentangles training (pure text lyric generation) from inference (melody compiled into segmentation + rhythm decoding constraints). It accepts content control inputs — title, genre, salient words — i.e., exactly the "theme/style/keywords" axis you want, and conditions on melody via durations (long/short note alignment) at decode time. Repository: github.com/amazon-science/unsupervised-melody-to-lyrics-generation, licensed **CC-BY-NC-4.0 (non-commercial)**. Caveat: the repo released materials/artifacts but is not a single packaged downloadable fine-tuned weight set in the SongComposer sense; the melody is a decode-time constraint, not a model input tensor. Fit: ~65% conceptually, but non-commercial license and constraint-based (not input-based) melody handling. [arXiv](#)

REFFLY (NAACL 2025, UCLA/Amazon). Melody-constrained lyric *revision/editing*: takes a melody (decomposed into pitch/duration/offset, prominent-note detection) plus an arbitrary plain-text draft (keywords, theme, or full text) and revises it into singable, melody-aligned, structured lyrics with sentence-level semantic control. This is arguably the *closest in spirit* to "note sequence + prompt → lyrics," because the "prompt" is the draft/theme text and the melody is an explicit input. However, I found no released model weights — it is a published framework/paper with synthetic-data training described, but no HuggingFace/GitHub weight artifact located. Fit: ~75% by design, but not open-weight as of this research. [The Moonlight](#)

SeqGAN-based melody-conditioned generators (e.g., "Melody-Conditioned Lyrics Generation with SeqGANs," arXiv 2010.14709; themed MC-SeqGAN). Take a melody line (syllable-note tuples) plus an optional theme embedding and emit a line of lyrics. Conceptually a near-exact match (melody + theme → lyrics), but these are small from-scratch GANs with weak text quality and generally no maintained open weights. Fit: ~50% conceptually, low practical quality.

Bucket (a): Lyric → Melody (inverse; for completeness)

- **TeleMelody** (EMNLP 2022, Muzic): two-stage lyric→template→melody; controllable via template (tonality, chord, rhythm, cadence). Code open. Wrong direction.
- **ReLyMe** (ACM MM 2022): adds music-theory lyric-melody relationship constraints on top of TeleMelody/SongMASS. Wrong direction.
- **SongGLM** (AAAI 2025, arXiv 2412.18107): lyric→melody with 2D alignment encoding, OctupleMIDI-style compound tokens, multi-task pretraining on a dataset of exactly 206,884 English MIDI songs totaling 4,921.79 hours of melodies. Wrong direction but excellent reference for symbolic tokenization schemes.
- **CSL-L2M** (arXiv 2412.09887): controllable song-level lyric→melody. Wrong direction.

Bucket (d): End-to-end audio song generation (the inverse you should NOT confuse with your goal)

These are the headline 2025–2026 open-weight song models. **All take lyrics + a style/genre text prompt as INPUT and emit AUDIO** — the opposite of producing lyrics, and they do not accept a symbolic melody.

- **YuE** (HKUST/M-A-P, 2025): lyrics2song foundation model; lyrics + genre text prompt + optional audio reference → full vocal+accompaniment audio. Stage-1 7B + stage-2 1B. License Apache-2.0 (weights); 24 GB GPUs can run ~2 sessions, full songs want 80 GB. Inverse direction. ([GitHub](#)) ([GitHub](#))
- **ACE-Step / ACE-Step v1.5** (ACE Studio + StepFun, 2025-2026): diffusion+LM hybrid; text prompt + lyrics → 48 kHz audio. Per the v1.5 paper (arXiv 2602.00744, submitted 31 Jan 2026): "extremely fast—under 2 seconds per full song on an A100 and under 10 seconds on an RTX 3090. The model runs locally with less than 4GB of VRAM" (vendor-reported). License is **MIT** for the ACE-Step weights (HuggingFace card shows "License: mit"; the bundled Qwen3-Embedding-0.6B text encoder is Apache-2.0), and the model is LoRA-trainable. Notably, its internal LM *generates* lyrics and metadata via chain-of-thought as a planning step, and it has lyric-editing (flow-edit) features — but it is not a symbolic-melody→lyrics tool. Inverse direction (with a lyric-planning side capability).
- **DiffRhythm** (2025): fast latent-diffusion full-song generation. Inverse direction.
- **SongGen** (ICML 2025, arXiv 2502.13128): single-stage autoregressive transformer; lyrics + descriptive text (instrumentation/genre/mood/timbre) + optional 3 s reference voice → song audio (mixed or dual-track). Open weights, code, data released. Inverse direction. ([arXiv](#))

Other referenced systems

- **DeepRapper** (ACL 2021, Muzic): Transformer rap generator modeling rhyme + rhythm (beat tokens), generating lyrics in reverse order. Pretrained weights downloadable. It models *beat/rhythm* but not an input melody note sequence or a controllable theme prompt — it generates rap from scratch with beat structure. Fit: ~30%. ([GitHub](#))
- **MuseCoco** (arXiv 2306.00110, Muzic): text→symbolic MIDI via attribute bridge. No lyrics at all. Not relevant except as a tokenization/attribute-control reference; text2attribute model is on HuggingFace ([XinXuNLPPer/MuseCoco_text2attribute](#)). ([Hugging Face](#))
- **ChatMusician** (arXiv 2402.16153, M-A-P): LLaMA2-7B continually pretrained on ABC notation; text/chords/melody/motif → symbolic music. Treats music as ABC text. It can be conditioned on a melody (ABC) and text, but it is oriented to composing *music*, not generating *lyrics* aligned to a melody. Fit: ~25%, but interesting because ABC is a clean text representation you could adapt. ([arXiv](#))
- **VersBand / LyricBand / MelodyBand** (arXiv 2504.19062): a prompt-controlled multi-task song framework. LyricBand is a QLoRA fine-tune (4-bit quantization + low-rank adapters, ~4k steps) of the bilingual Qwen-7B that generates complete lyrics from theme/emotion/genre text prompts (but *not* melody-conditioned). MelodyBand generates notes from text+lyrics. Demos and code referenced ([versband.github.io](#)), but lyric generation here is text-prompt-only, not melody-aligned. Fit: LyricBand ~35% (prompt control yes, melody conditioning no). ([arxiv](#))
- **SingSong / DiffSinger / M4Singer / OpenCpop**: singing-voice synthesis and datasets — they take lyrics+notes and produce *audio/acoustics*, not lyrics. M4Singer (~700 Chinese songs) and OpenCpop are datasets useful for fine-tuning alignment, not lyric generators. ([arxiv](#))

Technical notes for adaptation (for your homelab)

- **SongComposer tokenization** is the most directly reusable: note pitch tokens (12 pitch classes $\times$ 10 octaves = 120), 512 log-spaced duration bins clipped to $[-0.3, 6]$ s via $f(x) = \log(x+1)$, word-level tuples joined with $(\mid)$, multi-note words supported ($(\text{景}, <\text{A}\#3> <\text{F3}>, <\text{181}><\text{137}>, <\text{79}>)$). To convert your MIDI/note sequences, map note durations through the same log-bin formula and pitches to scientific notation, then concatenate with your theme text in the single prompt string. (arxiv)
- **Adding theme conditioning** to SongComposer M2L is a textbook LoRA/QLoRA task on a 3090/A6000: build (melody-tuples + theme instruction $\rightarrow$ lyrics) triples (you can synthesize themes from existing lyric corpora with an LLM, à la SongComposer's own GPT-4 song-summary pipeline), and fine-tune the M2L instruction distribution to honor a theme slot. The single-string API and InternLM2 chat format make this straightforward; serve via vLLM/SGLang afterward (InternLM2 is supported). (arxiv)
- **Constrained-decoding alternative (most license-clean, fully controllable):** Take any permissive open LLM (Llama-3, Qwen2.5, Mistral) you already self-host, do a Lyra-style plan-and-write: (1) prompt for a content plan from your theme/keywords, (2) decode lyrics under hard constraints derived from your note sequence — syllable counts per line = note counts per phrase, stressed-syllable placement aligned to long/prominent notes, phrase segmentation = melodic phrase boundaries. Implement constraints with a syllable counter (e.g., CMUdict/g2p) + a logit-processor or beam filter in (transformers)/vLLM. This gives you exact melody alignment + full prompt control with no model-license entanglement, at the cost of building the constraint compiler. REFFLY and Lyra are the design references.

Recommendations

Stage 1 — Validate the closest open-weight model first (days). Pull (Mar2Ding/songcomposer_sft), run it half-precision on your 3090. Test (a) pure M2L on your symbolic melodies and (b) experimentally concatenating a theme into the M2L prompt string. *Decision threshold:* if appending a theme already biases content acceptably while preserving alignment, ship it. If theme is ignored (likely, since it is untrained), proceed to Stage 2.

Stage 2 — Fine-tune SongComposer for melody+theme $\rightarrow$ lyrics (1-3 weeks). Build a few-thousand-example QLoRA dataset of (melody tuples + theme instruction $\rightarrow$ aligned lyrics), synthesizing themes from a lyric corpus with an LLM. Fine-tune on the A6000. This is the highest-ROI path to an actual exact-match model with permissive licensing (Apache-2.0 code, commercially usable weights). *Threshold to prefer this over Stage 3:* if SongComposer's base alignment quality and English text quality are adequate in Stage 1.

Stage 3 — Build a constrained-decoding pipeline if you need maximum control or cleaner licensing (2-4 weeks). Use a permissive LLM you already host (Qwen2.5/Llama-3) + a syllable/note-constraint compiler in the Lyra/REFFLY mold. This decouples lyric quality (use the best general LLM) from alignment (hard constraints), gives full theme/style/keyword control, and avoids any research-only license. *Prefer this if you find SongComposer's text quality limiting or want commercial certainty without weight-license caveats.*

Do NOT reach for YuE/ACE-Step/DiffRhythm/SongGen for this task — they are the inverse (lyrics+style → audio) and cannot consume a symbolic melody or emit standalone lyrics, though ACE-Step's internal lyric-planning LM could be a tangential source of draft lyrics.

Avoid depending on REFFLY expecting downloadable weights (none located) and treat Lyra's code as CC-BY-NC (non-commercial) — fine for prototyping, not for a shipped commercial plugin.

Caveats

- *Licensing nuances:* SongComposer's code is Apache-2.0 and its model card states weights allow free commercial use, but it is built on InternLM2-7B — verify InternLM2's base-model license terms for commercial deployment. Lyra is explicitly CC-BY-NC-4.0. YuE moved from CC-BY-NC to Apache-2.0 in Jan 2025 (some mirror repos still show the old CC-BY-NC text). ACE-Step weights are MIT (text encoder Apache-2.0). Always re-check the live license before shipping. [GitHub](#)
- *"Open weights" verification:* REFFLY and the SeqGAN themed generators are described in papers but I could not locate released, downloadable weight artifacts; treat them as designs, not deployable models, until you confirm a repo.
- *Quality ceiling:* Symbolic M2L models (SongComposer, SongMASS) are strongest in Chinese and have more modest English fluency than frontier general LLMs; the constrained-decoding route generally yields better English text quality.
- *Direction confusion is the dominant failure mode in this space:* the most visible 2025–2026 "open song models" are all lyrics→audio. If a tool's headline says "text/lyrics → song," it is the inverse of your requirement.
- *Recency:* ACE-Step v1.5 figures come from the project's own arXiv paper (submitted 31 Jan 2026), GitHub and Substack materials, and are partly self-reported benchmarks; the XL 4B-DiT series was released 2 Apr 2026 per the GitHub changelog. Treat performance figures as vendor claims pending independent evaluation.